On Graph Design for GNN-Based Network Anomaly Detection

Killian Cressant, Federico Larroca, Stefano Secci, and Pedro Braconnot Velloso, *Member, IEEE*

***Abstract*—Graph Neural Networks (GNNs) have gained significant attention for multivariate time series analysis in recent years. However, applying them to real-world networking data introduces several key challenges, particularly in designing meaningful and effective graph structures. In this paper, we propose a novel method for constructing initial graphs tailored for time series anomaly detection in complex network environments. Our approach, called COSI (COrrelation SImilarity), leverages two fundamental properties of real-world data: feature name semantics and statistical correlation. By combining natural language processing (NLP) with correlation analysis, COSI produces graph structures that significantly enhance GNN model performance for anomaly detection tasks, outperforming conventional graph construction methods in almost all evaluation scores across all datasets. We extensively evaluate COSI on three datasets, including two real-world networking datasets and one widely used benchmark dataset, and we open source the implementation to encourage reproducibility, further research, and practical adoption by the community.**

***Index Terms*—Time series, networks, anomaly detection, GNN, graph structure.**

I. Introduction

THE increasing complexity of modern networked systems, especially 5G and next-generation cellular infrastructures, poses significant challenges for effective network management [1]. These environments are characterized by high data volume, complex topologies, heterogeneous architectures, and diverse service demands. As a result, ensuring high levels of reliability, performance, and security becomes increasingly difficult. Among the critical challenges is the timely detection and mitigation of anomalies, which may arise from faults, intrusions, misconfigurations, or unexpected traffic behaviors [2]. Therefore, automating anomaly detection (AD) becomes essential in complex network environments for ensuring service reliability and security, as it enables early identification of unusual patterns that may indicate cyberattacks, faults, or performance degradation. In addition, with the high scales of 5G (and beyond) infrastructures, proactive detection helps maintaining low latency, high availability, and efficient resource utilization.

Nevertheless, one of the main challenges in detecting anomalies in such systems is how to cope with a significant amount of data composed of a large number of time series originating from different network/system components across diverse locations, often exhibiting complex spatial and temporal correlations [3]. In this context, traditional detection methods, typically based on rule-based logic or statistical models, usually fall short in handling the scale, diversity, and dynamism of today's networks. They struggle to detect subtle or evolving anomalies and often rely on assumptions impractical for real-time applications [4]. Hence, in this paper we focus on realistic networking anomalies that usually concern multiple time series simultaneously.

Consequently, there is a growing demand for more adaptive, scalable, and intelligent approaches to anomaly detection in network management [5]. A number of machine learning-based methods have been proposed to address these limitations [6], [7]. More recently, Graph Neural Networks (GNNs) [8], [9] have emerged as a promising solution for capturing the intricate spatial and temporal dependencies across large sets of time series — dependencies that traditional models such as Long Short-Term Memory (LSTM) and AutoEncoders often fail to detect effectively [3].

Graph Neural Networks (GNNs) are a powerful paradigm for learning from graph-structured data and appear particularly well-suited for modeling networked systems, which inherently exhibit relational structures among their components. By capturing both the interactions and contextual relationships between entities, GNNs provide a robust framework for learning complex behavior patterns in networks. However, a key issue in applying GNNs lies in constructing an appropriate graph structure that accurately models the underlying relationships within the system data [10]. One basic approach involves manually defining the graph based on domain knowledge or predefined metrics. Alternatively, graph learning techniques aim to infer automatically the graph structure from data; the main challenge in this context is identifying a sparse adjacency matrix that best captures the real dependencies among the components, while maintaining computational efficiency [11].

In this work, we propose COSI (COrrelation & SImilarity), a method to construct a graph adapted for GNNs and specifically designed for anomaly detection in modern networks with a high number of time series. Network data

Received 21 September 2025; revised 20 March 2026; accepted 13 April 2026. Date of publication 17 April 2026; date of current version 23 April 2026. This work was funded by CHIST-ERA Graphs4Sec (ANR-23-CHR4-0010), France 2030 INFLUENCE (DOS0192883) and IPCEI ME/CT (DOS0239248) projects. The associate editor coordinating the review of this article and approving it for publication was D. M. Manias. *(Corresponding author: Killian Cressant.)*

Killian Cressant, Stefano Secci, and Pedro Braconnot Velloso are with the CEDRIC Laboratory, Conservatoire National des Arts et Métiers (CNAM), 75003 Paris, France (e-mail: killian.cressant@cnam.fr; stefano.secci@cnam.fr; pedro.velloso@cnam.fr).

Federico Larroca is with the Universidad de la República, Montevideo 11300, Uruguay (e-mail: flarroca@fing.edu.uy).

Digital Object Identifier 10.1109/TNSM.2026.3684653

1932-4537 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

often lack interpretable graph topology, or provide topology that is too noisy to be directly useful for GNN training. In such cases, GNNs can be initialized with a randomized structure and subsequently refined using domain-specific insights when available [8]. We introduce a general-purpose framework designed to support the learning process of GNNs in network anomaly detection across varied network environments with a novel graph construction methodology. The key idea is to leverage both the semantic information embedded in feature names derived from the operating system and applications, and the statistical correlations in the system data to construct the graph structure. This approach is particularly well-suited to networking scenarios, where feature names are often intrinsically meaningful and tend to exhibit strong correlations with other features, that share similar naming conventions or are linked by protocols stack interactions. In a previous version of this work, we presented preliminary results using a single dataset, comparing our approach to various graph structures and an LSTM-based solution [12]. In this paper, we enhance our graph construction methodology to achieve the same results twice as fast. We further extend our evaluation to three datasets. On the widely used SWaT benchmark, the proposed approach consistently surpasses conventional graph construction methods, yielding up to a 4% improvement in a composite score (defined as the product of standard machine learning metrics) when employed with a vanilla GNN model, and up to 7.5% when combined with modern GNN-based anomaly detection algorithm GDN [8]. We also compare our framework with other state-of-the-art GNN models and analyze additional performance metrics. The main results demonstrate that using the edge selection provided by COSI, which retains only the most important edges, consistently improves efficiency and outperforms other graph structures and models across all datasets for most of the evaluated metrics.

The remainder is organized as follows. Section II describes basic information on AD techniques. In Section III, we present the related work focusing on GNN models applied to AD. Section IV defines our main design goals. To understand the requirements, we summarize the main characteristics of used datasets in Section V. We explain in detail the COSI graph construction in Section VI. Section VII analyses main results. Finally, we draw our conclusions and present the future work in Section VIII.

II. Background

An anomaly is characterized by a value or a group of values that significantly deviates from what is considered normal or expected behavior. From a statistical perspective, detecting an anomaly involves finding a data projection in which atypical patterns are more easily distinguished from normal behavior. In high-dimensional spaces, however, this task becomes more complex and often requires machine learning models to effectively identify relevant projections. Since anomalies are rare events, collecting a sufficient amount of labeled data for supervised learning is difficult. Additionally, in many real-world situations, it is challenging to clearly define what qualifies as an anomaly, which makes labeling even more difficult and inconsistent [13].

In earlier approaches, AD was often addressed using semi-supervised learning. Such methods tend to train models primarily on data assumed to represent normal behavior, while evaluation is conducted on datasets that include both normal and anomalous samples in a balanced manner, as in [14]. Traditional techniques rely on distance measures or density estimation to identify outliers. Anomaly detection has also been extensively studied in the context of time series. In the following, we focus on this type of data, which is particularly common in monitoring environments, such as network anomaly detection. With the rise of deep learning, new methods emerged for modeling time series, such as LSTM, Gated Recurrent Units (GRU), and other types of Recurrent Neural Networks (RNNs). For anomaly detection, these models are typically used to forecast future values, and anomalies are identified by measuring the discrepancy between predicted and observed values. This is commonly referred to as the prediction-based approach [15], [16].

Another common technique uses autoencoders in combination with models like LSTM. In this case, the system tries to reconstruct the original time series from a compressed internal representation. When the reconstruction fails to match the actual data, the deviation is flagged as an anomaly. This is known as the reconstruction-based approach [17], [18].

All of these approaches are built on the same principle: modeling normal behavior and applying a scoring mechanism to estimate the likelihood of an anomaly. However, in some scenarios, such as with network traffic data, unusual data load is not necessarily an anomaly, as for instance a peak in data traffic can be related to a legitimate situation, like a public demonstration that concentrates a great number of users in a specific region of a city. As shown in [19], the reconstruction error in such cases may not be sufficient to distinguish anomalies, and simple thresholding becomes ineffective. A further limitation of these methods is their struggle to capture long-term dependencies in data [20].

A graph structure can be described by $\mathbf{G} = (\mathbf{V}, \mathbf{E})$, a set of Vertices $\mathbf{V}$, a set of Edges $\mathbf{E}$ and, given an order on the vertex, an adjacency matrix $\mathbf{A}$ representation of the graph. In time series analysis, the initial graph structure often assumes a fixed number of nodes N, each representing a vertex, and each vertex can be associated to a 1-D time series. In such cases, for all $i \in [0, N]$, we have $\mathbf{x_i}$ that is an observed window of T observations. Hence, a fundamental problem is determining the set of edges $\mathbf{E}$, or the adjacency matrix $\mathbf{A}$, a task commonly referred to as association network inference. In the absence of explicit topological information, most approaches for inferring $\mathbf{E}$ rely on constructing a similarity function $S : \mathbf{V} \times \mathbf{V} \to \mathbb{R}$, such that $S(i, j) = s_{i,j}$. This similarity measure S can be based on correlation (total or partial) [3], [21], distances [9], or spectral properties such as eigenvalues [22]. Then, we can use a threshold to create a simple binary graph structure, such that: $S = 1$ if $s > T$, otherwise 0. Alternatively, we can keep the variation on S above the cut T. This function S, used for each pair of nodes gives us the adjacency matrix $\mathbf{A}$ of $\mathbf{G}$, according to the order of vertex used.

When applying graph structures to Graph Neural Network (GNN) models, specific properties of the adjacency matrix

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

are often preferred. Common desirable properties include sparsity, smoothness, and low-rank structure [23]. For instance, favoring a sparse adjacency matrix can improve computational efficiency and, in some cases, enhance model performance [24], [25]. On the other hand, smoothness and low-rank properties are typically preferred when aiming to capture more realistic and meaningful relationships in the data, especially those observed in real-world networks. These properties can be satisfied through two distinct strategies, hereafter referred to as post-optimization and in-process optimization, respectively.

- *Post-optimization:* after building an initial adjacency matrix, a post-processing step such as pruning can be applied to reduce the number of edges [24]. A common approach involves using a $top(K)$ function, which retains only the K strongest connections per node based on a chosen similarity metric. While this technique is widely used due to its simplicity and ease of implementation, it often yields poor performance when applied in isolation. This is primarily because the edge selection can be arbitrary, potentially discarding important connections [23].
- *In-process optimization:* a more general approach involves incorporating a regularization term directly into the optimization process. In this case, the problem can be formulated as finding an optimal adjacency matrix $\mathbf{A}^*$ such that:

$$\mathbf{A}^* = \underset{\mathbf{A}\in\mathbf{C}}{\operatorname{argmin}}\{L_{data}(\mathbf{A}, \mathbf{X}) + \lambda L_{reg}(\mathbf{A})\},$$

where X is the feature matrix, λ is a regularization hyperparameter and $\mathbf{C}$ represents the set of matrices that respect the constraints of the problem, L_{data} is the typical Loss based on the data such as reconstruction error while L_{reg} is the Loss of a regularisation term such as Lasso.

In the literature, we can also find constraints on the Laplacian of the graph, which is defined as $\mathbf{L} = \mathbf{D} - \mathbf{A}$, or the symmetric and normalized equivalent $\tilde{\mathbf{L}} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2}$ where $\mathbf{D}$ is the vertex degree diagonal matrix of the graph.

A GNN operates on graph-structured data by taking both an adjacency matrix, which defines the graph structure, and a feature matrix, which contains data for each node. GNNs typically consist of layers, like other neural networks, though they are often shallower due to challenges like over-smoothing [26]. Each layer is called a *message-passing* layer, where two key functions are applied: aggregation and update. The aggregation function combines feature values from neighboring nodes, while the update function transforms these aggregated values typically using a non-linear function. The exact form of these functions depends on the type of GNN, but this two-step process of aggregating neighbor information and updating node features is common across GNN architectures. In this paper, rather than focusing on the design of the two main GNN functions, we are interested in understanding the influence of graph structure on the performance of the GNN model.

III. Related Work

Graph Neural Networks (GNNs) have become increasingly popular for anomaly detection in multivariate time series, particularly in systems where dependencies between variables play a central role. In network environments, however, a fundamental challenge arises: unlike citation or road networks, no reliable ground-truth topology is typically available. As a result, graph construction becomes a critical design decision that directly impacts model behavior, convergence, and detection performance.

A. Graph-Based Models for Anomaly Detection

Early extensions of autoencoder architectures to graph domains include Variational Graph Auto-Encoders [27], which leverage graph convolutional layers to capture structural dependencies and infer latent relationships. These models were later adapted for anomaly detection [28], [29]. While effective in modeling spatial interactions, they do not explicitly capture temporal dynamics.

To address both spatial and temporal dependencies, several approaches combine graph convolutions with recurrent units such as GRU or LSTM cells [30], [31]. Other models integrate both aspects within a unified framework. For instance, the Graph Deviation Network (GDN) [8] predicts future time steps using attention mechanisms over a learned graph structure and measures deviations between predictions and observations to detect anomalies. Recently, models such as MST-GAT [32] have emerged, aiming to efficiently combine spatial and temporal dependencies for AD. Similar to GDN, MST-GAT begins with a fully connected graph and uses a *top-K* function to control graph sparsity. However, more recent models, such as CST-GL [21], claim superior performance; CST-GL also captures both spatial and temporal dependencies, but relies solely on correlation to identify the root cause of anomalies. Since correlation does not imply causality, this approach has limitations, and there is clear room for improvement in accurately determining causal relationships. These models demonstrate strong performance in anomaly detection tasks. However, their success remains closely tied to how inter-variable relationships are represented or initialized within the graph component.

B. Graph Structure Learning

Graph Structure Learning (GSL) seeks to infer or refine graph topology during training rather than relying entirely on predefined structures [33]. Several works initialize with random or fully connected graphs and iteratively update them as part of the learning process [11], [34]. These approaches suggest that adaptive graph refinement can improve traffic forecasting performances. Nevertheless, the role of the initial graph is often underexplored. Even when structure learning is employed, the starting topology influences optimization dynamics and the quality of the learned graph. This issue becomes particularly critical in domains where no natural topology exists.

C. Network-Specific Challenges and Graph Construction

In network systems such as telecommunication infrastructures, BGP routing environments, or industrial control systems, anomaly detection presents unique structural challenges. Unlike traffic forecasting tasks, where road topology

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

provides a natural graph, monitoring data in communication networks lacks a clear baseline topology [35]. Dependencies between components may be indirect, noisy, or partially observable.

As a result, several works construct task-specific graphs before training. For example, in malicious traffic detection proposed in [36], authors builds a graph based on flow interactions to detect unknown encrypted malicious traffic. In 4G/5G network monitoring, recent approaches construct graphs from radio topology information combined with K-nearest neighbour [37]. In [38], the authors propose a GNN-based framework for fault detection in 4G/5G networks using a bi-level graph structure: one graph at the Radio Access Network (RAN) level built from topological information, and a second temporal graph modeling software module execution. While the approach is conceptually appealing, it does not outperform baseline methods in their experiments. This highlights the difficulty of designing effective graph structures in complex networking environments, where interactions across system components may not align with predefined hierarchical assumptions. These methods highlight that graph construction is not merely auxiliary but central to performance.

However, such designs remain highly domain-dependent and may not generalize across datasets or operational conditions. In practice, correlation-based graphs, random initialization, or heuristic constructions are frequently adopted when no explicit topology is available. This variability underscores that graph construction itself is a key research problem in network anomaly detection.

D. Positioning of This Work

While prior studies demonstrate the effectiveness of spatial-temporal GNNs and adaptive graph learning, the impact of graph initialization in real-world network systems remains insufficiently explored. In particular, when no ground-truth topology is available, graph construction becomes a critical design choice.

Our contribution can be summarized as follows:

- We highlight the central role of graph initialization in network anomaly detection, especially in settings without a reliable baseline topology.
- We systematically analyze how the choice of graph structure influences both training efficiency and final detection performance.
- We propose **COSI**, a principled graph construction method designed as an alternative to random initialization and purely task-specific graph designs.
- We show that a carefully designed structure can significantly improve GNN optimization dynamics and anomaly detection accuracy in network systems.

IV. COSI Design Goals

A key consideration in designing graphs for GNNs is **sparsity**. A fully connected graph is typically uninformative, since when all nodes are interconnected, relational information becomes diluted and the model has no meaningful structure to learn from. Additionally, the computational cost of training GNNs increases more than linearly with the number of edges, meaning that denser graphs not only offer less discrimination power, but also lead to slower training and reduced performance [32], [39].

Historically, the concept of 'homophily', where connected nodes tend to have similar labels, has been widely used as an indicator of a graph's suitability for GNN-based learning. Numerous studies have shown that GNNs perform better when trained on homophilic graphs: while more recent research has begun to explore broader conditions, including various forms of heterophily and structural diversity [40], [41], the majority of current GNN architectures still perform best on graphs exhibiting homophilic patterns [41]. Assessing homophily in anomaly detection settings is, however, a complex task. In most network management datasets, node-level labels are not available, and the precise location of anomalies is unknown. For this reason, we treat **homophily** as an approximation criterion rather than a definitive measure.

In practice, obtaining an accurate and meaningful topology from raw network system data is often difficult. To address this, one strategy involves artificially increasing homophily, e.g., by connecting nodes that belong to the same metric group or are located in the same server. While this approach may increase structural coherence, it can also introduce unnecessary edges and potentially create misleading patterns of connectivity. Not all metrics are inherently related, even within the same group or hardware unit, so excessive connectivity may introduce artifacts rather than improve learning.

Since our task involves anomaly detection, it is crucial to construct a representation space that allows for effective separation between normal and abnormal patterns. While we can rely on the model to learn the most informative projection, we must also ensure that this projection is grounded in real relationships within the data. Without such grounding, the model might simply learn to separate data based on superficial differences, leading to overfitting or meaningless distinctions. To prevent this, we use the graph structure not just as a relational scaffold, but as a set of inductive biases that guide the model toward meaningful and interpretable projections. In addition, over-smoothing in GNNs demands the models to remain shallow [26]. In such settings, graph topology becomes crucial because information propagates over only a few hops. COSI therefore introduces direct connections between features that are meaningfully related even when their correlation is moderate. By combining correlation and similarity while keeping the graph sparse, COSI improves information flow and helps shallow GNNs better capture meaningful anomalies.

Anomaly detection is not only about identifying unusual behavior but also about uncovering the underlying causes. A critical aspect of this task is to provide a sense of **causality**, helping to localize faults and understand their origin. Therefore, the design of the graph structure should be guided by three key objectives in mind: (i) a projection space that remains simple and interpretable, (ii) sparsity to ensure computational efficiency, (iii) and a causal structure that supports root-cause analysis. To achieve these goals, we leverage all available information from the network system datasets, with particular emphasis on the feature names. In real-world monitoring data,

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

feature names often encode rich structural or hierarchical relationships. In the case of network systems, these names tend to follow systematic naming conventions that embed information about subsystems, metrics, protocol primitives, and locations, as detailed in Section V. We exploit this inherent structure to construct graphs that are sparse, homophilic, and causally meaningful. Edge selection based on these three properties appears to be the main factor contributing to our improved results compared to other methods across most datasets in Section VII.

V. Dataset and Graph Preprocessing

We consider three datasets collected from real-world systems, each consisting of multiple time series corresponding to different performance metrics. Each feature is associated with a semantically meaningful name that reflects its role within the system. These datasets are specifically designed for anomaly detection tasks and are structured to provide training sets containing only normal data, while the test sets comprise a mixture of normal and anomalous instances. To ensure a comprehensive evaluation, we include one classical industrial control dataset (SWaT) [42], one enterprise networking dataset (Cisco) [43], and one dataset from a 5G network environment (5G3E) [44].

A. SWaT Dataset

The SWaT dataset [42] is a real-world dataset collected from a water treatment system subject to cyberattacks. Although it does not originate from a telecommunication system, its extensive use in prior studies on anomaly detection provides a well-established baseline, thereby enhancing the validity of our experimental comparisons. Furthermore, the dataset exhibits several characteristics that make it particularly relevant to our analysis. It is comparable in scale to the Cisco dataset and contains time series features with statistical properties, such as partial correlations arising from the physical proximity of sensors that are analogous to those found in telecommunication systems. Most importantly, the dataset includes semantically informative feature names, which can be exploited for graph construction.

B. Cisco BGP Dataset

The Cisco BGP dataset [43] is used to test real-time BGP anomaly detection on real-world scenarios. The data was collected from a controlled network topology that correspond to a cloud service provider's datacenter. To ensure a more realistic dataset, the authors used real protocols, equipment, and typical traffic patterns. Authors produced different datasets with distinct traffic patterns (mostly video) from 500 Gbps to 1 Tbps: we use the one for training at 500 Gps and the test set at 1 Tbps. The motivation behind this choice is twofold; first, the test set is similar to the one the authors employed to evaluate their own framework; second, the difference in traffic volume is intentionally introduced to highlight a fundamental characteristic of the dataset, i.e. it is designed to support anomaly detection not based on traffic variations, but rather on fine-grained performance indicators within the system. As we will show, conventional unsupervised methods are prone to misclassification in the presence of traffic volume shifts, and often fail to accurately isolate true anomalies embedded in the dataset. Therefore, this evaluation scenario is designed to be more challenging, as it requires disregarding certain features and reasoning about the system's behavior at a global level. The test set includes 12 injected anomalies labeled for binary classification (normal vs. anomalous). At this stage, only anomalies of the BGP clear type have been considered in our experiments. Basically, a *BGP clear* restarts the BGP process on a given device. This operation is typically executed in response to routing inconsistencies, to "drain" a node before removing it from the forwarding plane for maintenance, or to address issues like memory leaks that affect shared memory and consequently disrupt BGP operations.

```
node_network_flags{device="calie63d0"}
node_network_iface_id{device="docker0"}
node_cpu_scaling_frequency_hertz{cpu="21"}
```

Fig. 1. Example of feature names in the 5G3E dataset.

C. 5G3E Dataset

In the 5G end-to-end emulation (5G3E) dataset [44], authors replay real mobile traffic dataset collected by a mobile operator into a 5G system testbed include core node functions and NS-3 real-time for emulating endpoints, using open-source 5G and radio access software. The dataset provides four feature groups across three distinct levels, each with a corresponding sampling rate. In this paper, we focus primarily on the physical layer. We also have 6 different servers, some of which belong to the 5G core network, while others cover RAN and endpoints and are distributed across four sites. Consequently, the dataset exhibits a structured organization that can be characterized as a block matrix: each block corresponds to an individual server, and within each block, further subdivisions represent different categories of performance metrics.

Four types of anomalies are injected in the dataset with different degrees of severity:

- **CPU overload** refers to a CPU stress on certain nodes across the network, ranging from 10% to 80%.
- **Link failure** injection between different sites.
- **Bandwidth limitation** limits the bandwidth setting to $(k \times (byteUp/Down)/duration) \times M$ with $k = 3$ and $M \in [2, 3, 4, 5]$
- **Packet loss** injected across the network with probability ranging from 10 to 80%.

While the first may correspond to certain attacks, the second and the third ones can correspond to network/system congestion events. After a thorough examination of the dataset, we excluded the test on CPU overload, as the corresponding anomalies were easily detected, even by traditional methods.

D. Datasets Characteristics

1) Features Names Encoding: All three datasets contains explicit knowledge in feature names. For the 5G3E, they correspond to Prometheus metrics names [45]. Figure 1 illustrates

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

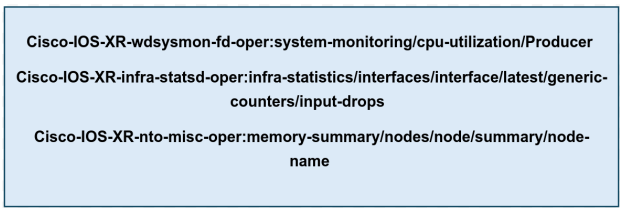


Fig. 2. Example of feature names in Cisco.

TABLE I
DATASETS DIMENSIONS. (1)PCK LOSS, (2)LINK FAILURE, (3)BANDWIDTH LIMITATION

Datasets	# Features	Train	Test	Anomalies (%)
SWaT	51	496,800	449,919	11.98
Cisco	54	15,738	29,222	1.12
5G3E	1065	576,000	300,000	$10.68^{(1)}\|0.47^{(2)}\|8.70^{(3)}$

an example of the kind of name we can find in the 5G3E dataset.

The word 'node' appears on more than 99% of the feature names of the dataset. 'Network' explains that the feature belongs to the group of network features, then 'scaling_frequency_hertz' describes what kind of measure it corresponds to. Curly brackets mean that the measure applies to that specific device. Hence, with this information, we have access to a symbolic representation of the system. For the Cisco dataset, the path is first and the last words correspond to the type of features, with a similar type of block classification as we can see in Figure 2.

Some works have already used Cisco feature names [46] to extract information on the data itself. But in this paper, instead of using the semantic information for ranking features that contain more explicit and useful human knowledgeable information, our key idea is to extract a graph structure from these feature names.

For the SWaT dataset, the features were only acronyms. However, we used the description of each features in the original journal [42] to make a complete semantic analysis. Each description was a short sentence that contains between 6 to 20 words.

2) Dimensions: Table I summarizes the main characteristics of the three datasets.

Beyond the scales in Table I, it is worth noting that the majority of 5G3E features exhibit minimal variability, with values that remain nearly constant over time. Another important aspect is the strong intra-group correlation, namely, features within the same group tend to be highly correlated, while inter-group correlation remains minimal, indicating that features across different groups are largely independent. This implies that the 5G3E dataset contains redundant information inside a cluster. Since the variance is also very low, we can infer that the noise is important, as often in real-world datasets. We also observe block or cluster patterns in the Cisco dataset, but with less impact than in 5G3E. Finally, we made classical preprocessing on all datasets, such as label encoding for Cisco, more details are available in the code.

VI. GRAPH STRUCTURE

Let the dataset $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N]$, be represented as a matrix of N time series. Each $\mathbf{x}_i$ is an univariate time series vector of the length T, with the assumption that $T \gg N$. We also suppose that each $\mathbf{x}_i$ has a unique feature name associated to it, represented by V_i in a vector space of feature names. An attributed graph of the dataset is then $\mathbf{G} = (\mathbf{X}, \mathbf{E})$ such that $\mathbf{X}$ is a set of nodes: a node feature matrix, with N nodes, and $(\mathbf{x}_i, \mathbf{x}_j) \in \mathbf{E}$ is a set of m edges, binary or weighted. We represent the set of edges by an adjacency matrix denoted by $\mathbf{A}$, such that $\mathbf{A}_{i,j} = w_{i,j}$; it can be either a weighted set of edges $\mathbf{A}_{i,j} \in [0, 1]$, or a binary matrix, in which 1 corresponds to an edge between i and j, and 0 otherwise. Hence, our goal is to build a graph $\mathbf{G} = (\mathbf{X}, \mathbf{A})$ that improves anomaly detection. We can see the impact of a spatial-temporal graph neural network as the generation of a signal $s = f(\mathbf{A}, \mathbf{X}, \boldsymbol{\theta})$, f represents the GNN and $\boldsymbol{\theta}$ the model parameters. We then train $\boldsymbol{\theta}$ to minimize a given loss function, typically by reducing the error in reconstructing or predicting the time series.

A. Graph Learned From A Smooth Signal

Kalofolias [47] introduced an in-process optimization approach to learn graph structures from smooth signals. This approach relies on the assumption that the underlying data exhibit a low smoothness score, formally expressed as the minimization of the term $\forall \mathbf{x} \in \mathbb{R}^N$, $tr(\mathbf{x}^\top \mathbf{L}\mathbf{x})$ where $\mathbf{L}$ is the graph Laplacian. This smoothness term quantifies the extent to which each data point exhibits similar behavior to its neighboring nodes in the graph. The author suggests to use an optimization of the adjacency matrix following this equation:

$$\min_{\boldsymbol{A} \in \mathcal{A}_m} \left(\|\boldsymbol{A} \circ \boldsymbol{Z}\|_{1,1} - \alpha \, \mathbf{1}^\top \log(\boldsymbol{A}\mathbf{1}) + \beta \, \|\boldsymbol{A}\|_F^2 \right). \quad (1)$$

where $\mathcal{A}_m$ is the space of symmetric adjacency matrix with null diagonal values, $\boldsymbol{Z}$ the pairwise distance matrix: $\boldsymbol{Z}_{i,j} = \|\mathbf{x}_i - \mathbf{x}_j\|^2$, $\circ$ is the Hadamard product, $\mathbf{1} = [1, \ldots, 1]^\top$ and $\|\cdot\|_F$ is the Frobenius norm. This formulation offers two advantages: first, it is a general solution compared to classical approaches that often rely on, Gaussian simplification hypothesis; in addition, they point out an interesting link between smoothness and sparsity, as enforcing smoothness inherently leads the resulting adjacency matrix to be sparse. However, a major limitation of this method lies in its reliance on the assumption that the smoothness term remains sufficiently small, which may not always hold. In this work, we aim at comparing the effectiveness of this graph construction technique applied to a GNN framework for anomaly detection.

B. Starting From Random Point

Another very common technique is to start with a similarity function as embedding, such as the cosine similarity function, then using a $TopK$ to prune the matrix and obtain a sparse matrix as input [8]. Nevertheless, this post-optimization approach presents some limitations. First, relying solely on the $TopK$ to enforce sparsity may lead to suboptimal results, as important structural information can be lost in the pruning process; for instance, high-degree nodes might be discarded

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

despite their potential contribution to the overall graph structure and predictive performance.

Moreover, most of this methods start from a randomized graph structure [34], before applying the $TopK$ operation. Due to the explosion of possibilities of graph structure, it is highly unlikely that the graph learned by the GSL module converges to a near-optimal graph structure. In contrast, building a representative initial adjacency matrix increases the probability for the GSL to converge towards a meaningful and effective structure, thereby improving the overall model performance. As a consequence, rather than starting at random as MTADGAT, we propose to first build a correlation matrix because it is a natural way to make a graph in signal processing [48], and then using features names to designed a matrix named COSI.

Algorithm 1 COSI Matrix Construction

Require: Input data X, threshold T, parameters λ and K, features names F
Ensure: COSI-based adjacency matrices A_4, A_5
1: $\mathrm{C}(X) \leftarrow \text{correlation}(X)$ # Compute correlation matrix
2: $W \leftarrow \text{split}(F)$ # Tokenize features names
3: $V_{\text{init}} \leftarrow \text{Word2Vec}(W)$ # Vectorize tokens
4: $V \leftarrow \sum_{i \in F_i} \text{IFS}(V_{\text{init}})$ # Aggregate embeddings
5: **for** each i, j in V **do**
6: $\quad \text{Cosim}(V_i, V_j) \leftarrow \frac{V_i^\top V_j}{\|V_i\| \cdot \|V_j\|}$
7: **end for**
Compute hybrid similarity matrices:
8: $\text{COSI}_4^{i,j} \leftarrow |\mathrm{C}(x_i, x_j)| \cdot |\text{Cosim}(x_i, x_j)|$
9: $\text{COSI}_5^{i,j} \leftarrow \lambda |\mathrm{C}(x_i, x_j)| + (1-\lambda)|\text{Cosim}(x_i, x_j)|$
10: **for** each pair (i, j) in the set of nodes **do**
11: $\quad$ **if** $\text{TopK}_i(\text{COSI}_M^{i,j}) < T$ **then**. # $M \in \{+, \times\}$
12: $\quad\quad A_M^{i,j} \leftarrow 0$
13: $\quad$ **else**
14: $\quad\quad A_M^{i,j} \leftarrow 1$
15: $\quad$ **end if**
16: **end for**
17: **return** A_4, A_5

C. COSI Graph Designing

The pseudo-code for the graph construction process is presented in Algorithm 1. Consistent with common practices suggested in prior studies [21], the initial step involves constructing a correlation graph. Hence, we calculate the Pearson correlation matrix to get a first graph (Line 1). Ideally, a graph would preserve causal relationships between features. As we are looking for the root cause and a sense of causality, we considered causal graphs. However, building such a graph can be computationally expensive and inefficient for large datasets like 5G3E. Accordingly, in this study, we opted to keep edges only when there is plausible evidence of causality. Instead of using such complex graphs, we used the semantic link between features names to construct another graph, named the similarity graph, and then we merged them to capture these relationships. Thus, whenever both correlation and a semantic link between feature names exist, a connection between the corresponding nodes is established. Therefore, the key idea of COSI relies on combining the semantic information embedded in feature names and the statistical correlations in the system data to construct the initial graph structure.

Hence, in our approach, we employ Natural Language Processing (NLP) methods using Word2Vec [49] and statistical tools to build node embeddings and the adjacency matrix. We use three different documents corpus, one for each dataset. For Cisco, we use only the standard pre-trained Word2Vec model, combined with the feature names, ensuring that all relevant terms appearing in the feature names are included for effective representation. For 5G3E dataset, we enrich the corpus with a documentation from Prometheus [45] that provides detailed explanations of the feature names. Finally, for the SWaT dataset, we use the feature descriptions provided in [42] to capture the full semantic meaning. This approach aims to improve the quality of feature representations by retraining Word2Vec on context-specific data, resulting in more semantically meaningful embeddings.

As every NLP process, the first tokenization step is important because it provides us specific information. For example in 5G3E, we keep '=' as a regular character rather than as a delimiter. This allows us to preserve more information in tokens and create a word like "$cpu = 1$", which retains the association between "cpu" and "1", instead of splitting them. In addition, the "cpu" word can also appear separately. We reach approximately 8,700 words for the 5G3E, around 970 for Cisco and 658 for SWaT. Once it is done, we retrain a Word2Vec model [49] to obtain a list of vectors for each word previously found. To ensure a more general graph construction, we use only non-domain-specific models in this work. Since our features consist of multiple words, we apply an aggregation function at this stage to generate a vector representation for each feature. A widely used technique is the term frequency-inverse document frequency (TF-IDF) method [50], which is well-suited for large datasets. However, because our feature set is relatively small and standardized, conditions under which Zipf's law may not apply, we avoid the logarithmic frequency scaling typical of TF-IDF. Instead, we use a straightforward frequency count (the "brute frequency variant") to preserve a sparser distribution in the resulting vector space. After this step, we obtain a list of vectors V_i, one for each feature, thus completing the node embedding process, illustrated in Lines 2–4 in Algorithm 1.

To finally create the adjacency matrix of the similarity, we use the classical cosine similarity function, given by Equation (2), to estimate the distance between each node (Lines 5–6).

$$Cosim(V_i, V_j) = \frac{V_i^T V_j}{||V_i|| \cdot ||V_j||}. \tag{2}$$

Hence, we can compare each feature in the vector space. As a consequence, we end up with two dense matrices: one for correlation and one for semantic. Next, we merge them using two different functions (Lines 8–9), leading to two graph structures $COSI_\times$ and $COSI_+$, given by Eq. 3 and Eq. 4.

$$S_{i,j} = |Corr(\mathbf{x}_i, \mathbf{x}_j)| \times |Cosim(\mathbf{x}_i, \mathbf{x}_j)|, \tag{3}$$

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

and

$$S_{i,j} = \lambda |Corr(\mathbf{x}_i, \mathbf{x}_j)| + (1-\lambda)|Cosim(\mathbf{x}_i, \mathbf{x}_j)|. \quad (4)$$

The product score in Eq. 3 expresses the idea of considering both correlation and similarity. This formulation represents a straightforward and natural method for defining a score within this context. Nevertheless, it tends to create really sparse matrices and is less flexible than the second one. The λ parameter in Eq. 4 enables us to control the balance between the graph's similarity structure and the underlying data. Since a single word may carry less informative value than others, depending on the dataset, we can adjust the λ parameter to reduce the impact of the similarity score on the resulting graph. This, in turn, increases the relative influence of the correlation component, which is directly derived from the data. It can be shown that when the data are already sufficiently separated, $COSI_{\times}$ appears advantageous. Conversely, in low-variance regimes, $COSI_{+}$ may be preferable.

D. The Pruning Procedure

While homophily and causality structure are achieved through the construction process, sparsity is not inherently achieved and must be explicitly enforced, since at the end of Line 9 the methods described yield fully dense matrices, which are computationally expensive to process. To satisfy the sparsity requirement, we employ two mechanisms to construct the final binary adjacency matrix. The first is a Top_iK operation, which selects the top K highest scores for each node i: this parameter controls concomitantly the sparsity level of the resulting graph and its homophily; specifically, when K is small the resulting graph tends to connect only nodes with similar scores in terms of both semantic meaning and correlation values. In such case, we achieve a high degree of homophily. Conversely, using a higher K value allows the inclusion of information from a broader set of nodes, potentially capturing more diverse relationships across different regions of the graph. However, relying solely on $TopK$ can be problematic, as it may retain connections with very low scores. To address this limitation, we introduce an additional threshold T, which filters out insignificant connections by discarding edges with scores below this value.

These two mechanisms can be applied individually or in combination, depending on the desired graph properties. When both are used together in a balanced manner, they offer greater control over the sparsity and quality of the adjacency matrix. For instance, setting a low threshold T with a small K ensures that each node retains exactly K edges, focusing on the top-ranked connections. On the other hand, using a high threshold T with a large K makes the threshold the dominant pruning mechanism, discarding all edges below the specified score level regardless of rank. In practice, we apply the $TopK$ mechanism to large datasets such as 5G3E, where certain nodes may have disproportionately high connectivity compared to others, requiring a lower threshold T to keep only a few meaningful edges. For smaller datasets like SWaT and Cisco, we fix $K = 100$ relying exclusively on the threshold T to control sparsity. This choice ensures greater consistency

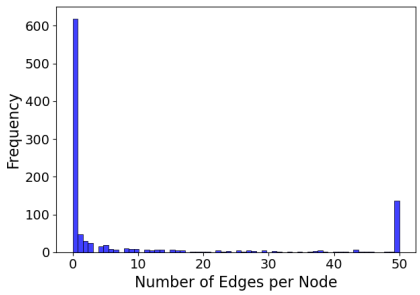


Fig. 3. Edges number distribution of $COSI_{\times}$ Top 50. (5G3E).

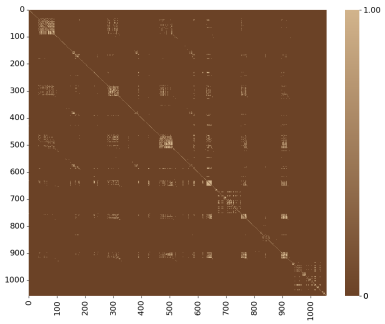


Fig. 4. Adjacency matrix of $COSI_{\times}$ graph - Th: 0.9.

in graph construction across nodes while preserving only the most relevant connections. The final adjacency matrix is created according to the following rules:

$$\mathbf{A}_{i,j} = \begin{cases} 0 & \text{if } Top_iK(S_{i,j}) \leq T \\ 1 & \text{if } Top_iK(S_{i,j}) > T \end{cases}$$

As a result, we obtain a sparse graph structure, as illustrated in Figure 3. The distribution exhibits a spike at 50 edges, as it aggregates all values that would otherwise appear beyond 50 in the absence of TopK. Without this artifact, the distribution would resemble the patterns typically observed in graph structures [33].

Figure 4 illustrates the *COSI* graph adjacency matrix and the block structure associated to the 6 servers. We can observe that one is very different than the others because it is the network core. Inside, we can see the different groups of metrics, with the most connected one being the CPU values.

VII. Graph Structure Evaluation

In this section, we describe the evaluation metrics and we present our results. First, we compare the performance of GNN with COSI and other models. Next, we evaluate the impact of $TopK$ on the training time. Finally, we show the influence of the graph structure on the performance of two GNN models. Our COSI implementation is publicly available[1] and all training was conducted on an AMD Ryzen Threadripper PRO 7955WX 16-core CPU.

A. Metrics for Anomaly Detection

The most common metrics used to evaluate the efficiency of GNNs models are the trio Precision/recall/F1-score [3], [8].

[1] github.com/Killian-cressant/COSI

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON PERFORMANCE GDN AND OTHER METHODS (RESULTS IN PERCENT)

Model	Packet loss			Bandwidth limitation			Link failure		
	F1 Score	Precision	Recall	F1 Score	Precision	Recall	F1 Score	Precision	Recall
LSTM-AE	38.85	24.74	90.44	96.24	99.76	92.96	**98.50**	**97.69**	**99.32**
Random graph	99.63	99.40	99.86	99.50	99.19	99.80	93.85	91.26	96.59
Correlation graph	**99.87**	**99.80**	99.94	98.43	96.95	**99.96**	97.32	95.48	99.24
$COSI_{+}$	99.13	98.86	99.38	99.37	99.38	99.36	96.39	93.64	**99.32**
$COSI_{\times}$	99.79	99.63	**99.96**	**99.91**	**99.91**	99.90	96.32	93.57	99.24

Model	Cisco			SWaT		
	F1 Score	Precision	Recall	F1 Score	Precision	Recall
LSTM-AE	4.86	2.60	**36.88**	85.36	**99.78**	74.58
Random graph	9.61	6.59	17.73	87.06	99.64	77.30
Correlation graph	22.32	21.95	22.70	87.93	99.24	78.94
$COSI_{+}$	28.07	**41.21**	21.28	87.98	99.41	78.91
$COSI_{\times}$	**28.35**	37.07	22.98	**88.46**	99.22	**79.80**

However, previous studies show that these metrics are not particularly suitable for time-series AD evaluation [51]. Therefore we decided to use F1-score for benchmark comparisons, as it is well suited for imbalanced datasets. Then we also use the AUC (Area Under the Curve)-ROC curve [21], [36]. Moreover, we use a *time-ranged* Precision/Recall/F1 [52], to provide a better estimation when dealing with time series and ranged anomalies, but they still rely on a threshold that is selected based on the validation set.

$$Recall_{\rho}(R,P) = \frac{\sum_{i=1}^{N_r} Recall_{\rho}(R_i,P)}{N_r},$$

where all R_i form a set of real anomaly ranges, P a set of predicted anomaly ranges and N_r the total number of real anomalies.

$$Precision_{\rho}(R,P) = \frac{\sum_{i=1}^{N_p} Precision_{\rho}(R_i,P)}{N_p},$$

where N_p the total number of predicted anomalies. Finally, we also used Volume Under Surface (VUS) metrics: VUS-AUC and VUS-PR (Precision Recall) [51]; these metrics are specifically designed for anomaly detection in time series, they extend the AUC-ROC/PR to the ranged versions previously described, and also add a third dimension for batch size. It results in a score that is the volume under a 2D-surface.

B. GNNs Models Comparison

We use an LSTM autoencoder as a classical baseline for time-series anomaly detection to compare traditional methods with GNN-based models. The LSTM model was fine-tuned via grid search over key hyperparameters (optimizer, number of layers, and hidden dimensions) to ensure competitive performance. We then evaluated the GDN model using a randomly initialized graph structure. To account for randomness, experiments were repeated with different seeds, and the best score for each dataset was reported. When feasible, we additionally evaluate partial versions of COSI using only correlation or only similarity rather than their combination.

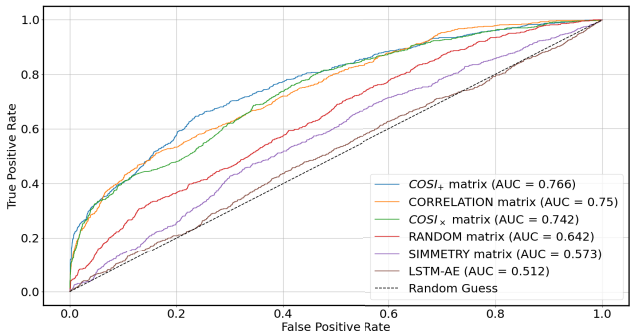


Fig. 5. AUC ROC curve results for Cisco dataset.

Experiments were conducted on the 5G3E dataset (across multiple anomaly types), as well as on the CISCO and SWaT datasets. Performance was evaluated using F1-score, Precision, and Recall, with the GDN model.

Table II presents the results. They suggest that while LSTM-based models can outperform GNNs for certain types of anomalies, they may completely fail to detect anomalies that present strong spatial dependencies. In contrast, COSI remains efficient across all datasets and anomaly types. In this test, the λ of $COSI_{+}$ was not tuned, and only $\lambda = 0.5$ was used, which may explain its poorer results compared to $COSI_{\times}$. Nevertheless, both outperforms in the majority of cases randomness, LSTM, and the correlation graph. We can also observe that, in some cases, the correlation graph alone achieves very good performance, while in others it falls below COSI. This illustrates the main limitation of correlation: sometimes, it is merely the symptom of noise and should be ignored.

With the Cisco dataset, smaller than 5G3E, we could evaluate the performance of $COSI_{+}$ with a tuned λ against other adjacency matrix using the GDN model: as shown in Fig. 5, the $COSI_{+}$ matrix clearly outperforms the alternatives, achieving an AUC-ROC of up to 0.76; in contrast, the LSTM-AE performs only marginally better than random, and the random adjacency matrix achieves a lower AUC-ROC

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

TABLE III
TIME REQUIRED FOR TRAINING: LSTM-AE VS GDN VARIANTS

	LSTM- AE	Classic top20	$COSI_{\times}$	$COSI_{+}$ λ= 0.5	Correlation	Classic top60	Similarity
Training time	1h	8h17	11h52	13h30	14h	20h30	105h
Graph construction time	-	∼1min	30min	30min	25min	∼1min	5min

TABLE IV
EFFECT OF TOPK WITH COSI MATRIX. *FOR PACKET LOSS ANOMALY

TopK	COSI model		Random Graph	
	Training time	F1 score*	Training Time	F1 score*
Top5	1h54	99.29	2h50	99.25
Top10	2h18	80.31	4h45	99.40
Top30	3h	96.35	11h53	99.41
Top50	4h44	98.34	18h54	99.42
Top 160	5h25	99.79	-	-
Without	11h52	99.79	-	-

of approximately 0.64. In addition, we observe that $COSI_{\times}$ performs better on the 5G3E dataset than an untuned $COSI_{+}$, whereas a tuned $COSI_{+}$ achieves superior results on the Cisco dataset. Since $COSI_{+}$ requires setting a hyperparameter λ, we believe that carefully tuning this parameter could further enhance performance, though it is also computationally expensive and time-consuming. Both COSI models can be used, depending on the available resources and time constraints.

The LSTM-AE model is confused by the variation in traffic load between the training and test sets, since it concentrates solely on temporal patterns in time series. Although the traffic load is intentionally altered across these sets, a robust anomaly detection framework is expected to remain unaffected by such fluctuations because, in practical scenarios, traffic load may vary naturally and does not necessarily indicate anomalies. This highlights an important limitation of LSTM-based approaches for AD in traffic forecasting: any variation in traffic load may trigger false alarms, even when the system is functioning normally. In contrast, GNN models capture spatial dependencies between features, which allows them to maintain robust performance even when temporal patterns are affected by changes in traffic volume.

C. Training Time Comparison and TopK Effect

We compare the training and construction times of the different models. Table III presents the results for the 5G3E dataset. Since the computational complexity of GNN models typically increases quadratically with the number of nodes, we first evaluate the COSI model without the TopK sparsification to ensure a fair comparison. As expected, LSTM models consistently exhibit significantly faster training times than GNN based models, regardless of the adjacency matrix's sparsity. Nevertheless, COSI achieves the highest performance on 5G3E while requiring only 12 hours of training, substantially less than the random Top60 model (used for the score comparison), which takes approximately 20 hours and yields inferior results.

On the 5G3E dataset, which is significantly larger than the other two, we can evaluate the importance of using a TopK operation. Table IV indicates that when we add too many nodes, the Graph Structure Learning component of the GDN framework struggles to explore the full range of structural possibilities, which results in a very slow convergence, even when we introduce more information, namely, additional edges. In contrast, when the COSI matrix is used as an initial structure, training becomes more efficient, as it not only applies a TopK constraint, but also eliminates irrelevant edges. In this case, training may be slightly less efficient with low TopK values, but it ultimately achieves better performance and reaches a strong Top-5 approximation more quickly. At TopK = 160, the model achieves results comparable to those obtained without any TopK constraint (using a threshold of 0.8), while the training time is reduced by more than half. In summary, applying a TopK constraint maintains the model performance while significantly reducing training time, when combined with an appropriate threshold on COSI matrices. The reason why TopK is so efficient in these cases is probably due to the redundancy of information in networking systems, where TopK reduces training time by removing edges in the adjacency matrix without sacrificing unique information.

D. Graph Structure Evaluation

To evaluate the influence of graph structure on the training of a GNN, we conduct a fair comparison by generating 100 random graph adjacency matrix. To properly evaluate this part, we need to train thousands of models using different sets of hyperparameters and model architectures. We chose to use only the SWaT dataset, as it is smaller but also widely used. This allows us to enable broader comparisons in the future while keeping the experiments computationally feasible. We use a random uniform function to place all nodes then apply a gaussian function for the weight based on the euclidian distance plus a sparsity parameter r:

$$W = \begin{cases} e^{-D/2\sqrt{\sigma}} & \text{if } > r \\ 0 & \text{else.} \end{cases}$$

We also set all diagonals entries to zero. In the experiment, r is set to 0.6 and σ to 0.5. We previously showed that sparsity remains a key property for achieving both good performance and reasonable training time. To this end, r and σ are set to reach approximately the same sparsity levels as COSI.

To preserve computational efficiency, we restrict training to three specifically designed graph structures. The first technique, proposed by Meinshausen and Bühlmann [53], employs Lasso regression to the covariance matrix of a graph in a sparse high dimensional context. The second method is the one proposed by Kalofolias [47], as mentioned in Section VI-A. Finally, we compare to our $COSI_{\times}$ method.

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

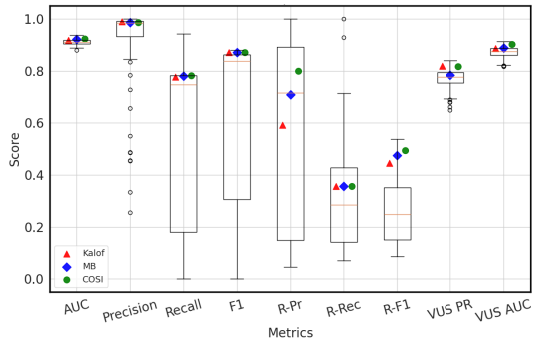


Fig. 6. Graph structure comparison using GCN.

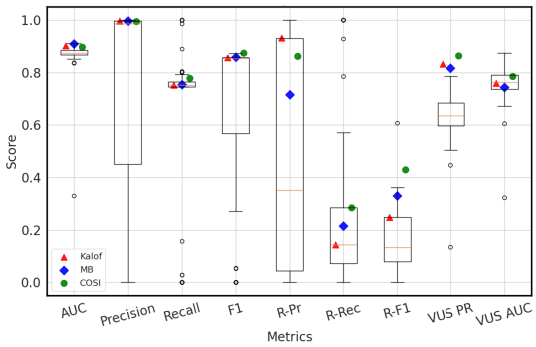


Fig. 7. Graph structure comparison using GDN.

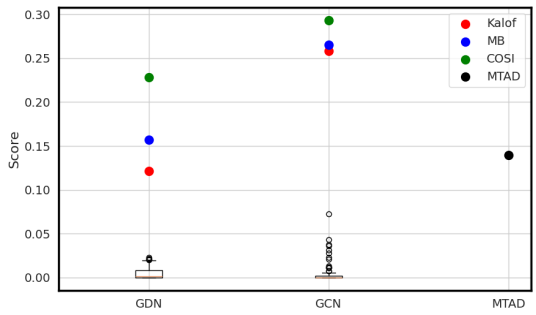


Fig. 8. Weighted evaluation with product metrics.

In this experiment, we train each of these matrices on multiple sets of hyperparameters on a vanilla graph convolutional network, Fig. 6 and then with GDN, Fig. 7 on the SWAT dataset, and finally compared with random graph with MTAD-GAT [34]. Next, we use the best hyperparameters set for each of the 3 matrices, and then we train each of the random graph with those 3 sets. In the end, we select the best results for each graph using the sum of all metrics. This manipulation help us to avoid the influence of hyperparmeters. As we can observe in Figure 6, the 3 designed matrices consistently outperform the median performance of the random graphs and, in most cases, exceed the third quartile. A similar results is obtained when we used GDN in Figure 7. This means that graph structure learning struggle to find the best possible matrices and that using our own remain a very important first step. For all metrics, they are among the best values, in particular for COSI which is almost always better than the others.

Nevertheless, we can observe that they are not necessarily the top performers across all graph configurations. In fact, relying on a single evaluation metric can be imprecise and potentially misleading, which is why we consider a multi-metric analysis. As a consequence, evaluating a single aggregated score is preferred. Therefore, we compute a composite score by taking the product of several normalized metrics: F1 score (ranged also), AUC, VUS-PR, and VUS-ROC. One may notice that we did not include precision and recall (even ranged values) to avoid redundancy with the F1 score, which is the harmonic mean of precision and recall. Although Figure 8 provides a comprehensive visual representation, it may be more challenging to interpret. In this figure, we also add the results of a random initialization graph structure model: MTAD-GAT [34]. Since we cannot modify the graph structure, we only get a single value. Results show that using different graphs structures will indeed change the way a GNN will be trained. We can see that COSI demonstrates overall strong performance compared to classical graph structures, with all methods performing far better than random structures. More importantly, the results show that using more advanced GNN models is not as critical as selecting an appropriate initial graph structure, since GCN scores are significantly higher with a well-designed graph structure than GDN scores.

However, establishing a clear theoretical or statistical justification for performance improvements remains a complex task and falls outside the scope of this paper. Nonetheless, our results demonstrate that employing a specifically designed graph structure, such as COSI, provides a substantially improved trade-off between performance and computational efficiency, achieving up to a $9\times$ gain in composite score over random graphs and more than a $1.4\times$ improvement over classical graph construction approaches, without introducing significant additional computational complexity compared to these methods.

VIII. Conclusion

In this paper we first show that carefully designing a graph structure has indeed an important impact on the GNN training efficiency. We highlight that even using more advanced GNN model such as GDN or MTAD is less important than choosing the right graph design below. Methodologically, we show that the COSI matrix is highly effective for training a GNN model, reducing training time without sacrificing detection performance. On SWaT and other network datasets, COSI consistently outperforms standard and random matrices, improving ROC AUC by up to 0.12 (Cisco) and 2.4% (5G3E), highlighting the importance of graph design over GNN choice. Finally, we developed a COSI model that contains TopK pruning methods to reduce training time. We showed that this methods is efficient to keep a low training time without compromising too much the efficiency when using with COSI model but not with random graphs.

Although our methodology partially leverages causal signals when constructing the graph structure, the resulting adjacency matrix remains undirected, which limits causal interpretation. In future work, we plan to extend the framework to directed graphs by employing GNN models specifically designed for directed structures and learning edge directions during

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

training. Such an extension could enable a more principled causal interpretation and improve root-cause analysis.

REFERENCES

[1] A. Mekrache, A. Ksentini, and C. Verikoukis, "Machine learning in FCAPS: Toward enhanced beyond 5G network management," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 4, pp. 2769–2797, 2024.

[2] A. Afaq, N. Haider, M. Z. Baig, K. S. Khan, M. Imran, and I. Razzak, "Machine learning for 5G security: Architecture, recent advances, and challenges," *Ad Hoc Netw.*, vol. 123, Dec. 2021, Art. no. 102667.

[3] K. Chen, M. Feng, and T. S. Wirjanto, "Multivariate time series anomaly detection via dynamic graph forecasting," 2023, *arXiv:2302.02051*.

[4] H. Huang, P. Wang, J. Pei, J. Wang, S. Alexanian, and D. Niyato, "Deep learning advancements in anomaly detection: A comprehensive survey," *IEEE Internet Things J.*, vol. 12, no. 21, pp. 44318–44342, Nov. 2025.

[5] M. Boudjelli, S. Cherrared, P. B. Velloso, X. Huang, F. Guillemin, and S. Secci, "DREAM: Dual foREcAsting model for network anomaly detection," in *Proc. NOMS - IEEE Netw. Operations Manage. Symp.*, May 2025, pp. 1–7.

[6] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab, "Machine learning for anomaly detection: A systematic review," *IEEE Access*, vol. 9, pp. 78658–78700, 2021.

[7] A. Dridi, C. Boucetta, S. E. Hammami, H. Afifi, and H. Moungla, "STAD: Spatio-temporal anomaly detection mechanism for mobile network management," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 1, pp. 894–906, Mar. 2021.

[8] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI*, vol. 35, 2021, pp. 4027–4035.

[9] M. Jin et al., "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10466–10485, Aug. 2023.

[10] M. Dong and Y. Kluger, "Towards understanding and reducing graph structural noise for GNNs," in *Proc. ICML*, 2023, pp. 8202–8226.

[11] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020.

[12] K. Cressant, P. B. Velloso, and S. Secci, "GNN graph structures in network anomaly detection," in *Proc. IEEE Netw. Operations Manage. Symp.*, May 2025, pp. 1–9.

[13] N. Goernitz, M. Kloft, K. Rieck, and U. Brefeld, "Toward supervised anomaly detection," *J. Artif. Intell. Res.*, vol. 46, pp. 235–262, Feb. 2013.

[14] P. Guansong, S. Chunhua, C. Longbing, and H. A. V. den, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, no. 2, p. 38, Mar. 2021.

[15] Y. Wei, J. Jang-Jaccard, W. Xu, F. Sabrina, S. Camtepe, and M. Boulic, "LSTM-autoencoder-based anomaly detection for indoor air quality time-series data," *IEEE Sensors J.*, vol. 23, no. 4, pp. 3787–3800, Feb. 2023.

[16] A. Hekmati, J. Zhang, T. Sarkar, N. Jethwa, E. Grippo, and B. Krishnamachari, "Correlation-aware neural networks for DDoS attack detection in IoT systems," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 3929–3944, Oct. 2024.

[17] A. Diamanti, J. M. S. Vilchez, and S. Secci, "LSTM-based radiography for anomaly detection in softwarized infrastructures," in *Proc. 32nd Int. Teletraffic Congr. (ITC)*, Sep. 2020, pp. 28–36.

[18] B. Lindemann, B. Maschler, N. Sahlab, and M. Weyrich, "A survey on anomaly detection for technical systems using LSTM networks," *Comput. Ind.*, vol. 131, Oct. 2021, Art. no. 103498.

[19] M. S. Elsayed, N.-A. Le-Khac, S. Dev, and A. D. Jurcut, "Network anomaly detection using LSTM based autoencoder," in *Proc. Q2SWinet*, 2020, pp. 37–45.

[20] J. Kim, H. Kim, H. Kim, D. Lee, and S. Yoon, "A comprehensive survey of deep learning for time series forecasting: Architectural diversity and open challenges," *Artif. Intell. Rev.*, vol. 58, p. 216, Apr. 2025.

[21] Y. Zheng et al., "Correlation-aware spatial–temporal graph learning for multivariate time-series anomaly detection," *IEEE Trans.on Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11802–11816, 2023.

[22] Z. Zhang, P. Cui, J. Pei, X. Wang, and W. Zhu, "Eigen-GNN: A graph structure preserving plug-in for GNNs," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2544–2555, Mar. 2023.

[23] I. A. Chikwendu, X. Zhang, I. O. Agyemang, I. Adjei-Mensah, U. C. Chima, and C. J. Ejiyi, "A comprehensive survey on deep graph representation learning methods," *J. Artif. Intell. Res.*, vol. 78, pp. 287–356, Oct. 2024.

[24] S. Qiu, L. You, and Z. Wang, "Optimizing sparse matrix multiplications for graph neural networks," in *Proc. ICML*, 2022, pp. 101–117.

[25] S. Qiu, L. You, and Z. Wang, "Optimizing sparse matrix multiplications for graph neural networks," 2021, *arXiv:2111.00352*.

[26] U. Alon and E. Yahav, "On the bottleneck of graph neural networks and its practical implications," in *Proc. ICLR*, 2021.

[27] T. N. Kipf and M. Welling, "Variational graph auto-encoders," 2016, *arXiv:1611.07308*.

[28] L. Zhang et al., "Self-supervised variational graph autoencoder for system-level anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2023.

[29] Y. Hu, A. Qu, and D. Work, "Detecting extreme traffic events via a context augmented graph autoencoder," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 6, pp. 1–23, Dec. 2022.

[30] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph WaveNet for deep spatial–temporal graph modeling," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 1907–1913.

[31] A. Chawla, A.-M. Bosneag, and A. Dalgkitsis, "Graph-based interpretable anomaly detection framework for network slice management in beyond 5G networks," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, May 2023, pp. 1–6.

[32] M. Wu, C. Zhu, and L. Chen, "Multi-task spatial–temporal graph attention network for taxi demand prediction," in *Proc. 5th Int. Conf. Math. Artif. Intell.*, Apr. 2020, pp. 224–228.

[33] D. Chen et al., "GSLB: The graph structure learning benchmark," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 30306–30318.

[34] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2020, pp. 841–850.

[35] Y.-M. Shin, C. Tran, W.-Y. Shin, and X. Cao, "Edgeless-GNN: Unsupervised representation learning for edgeless nodes," *IEEE Trans. Emerg. Topics Comput.*, vol. 12, no. 1, pp. 150–162, Jan. 2024.

[36] C. Fu, Q. Li, and K. Xu, "Flow interaction graph analysis: Unknown encrypted malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 2972–2987, Aug. 2024.

[37] P. Almasan, J. Suárez-Varela, A. Lutu, A. Cabellos-Aparicio, and P. Barlet-Ros, "Enhancing 5G radio planning with graph representations and deep learning," in *Proc. 3rd ACM Workshop 5G Beyond Netw. Meas., Model., Use Cases*, Sep. 2023, pp. 14–20.

[38] R. Bourgerie and T. Zanouda, "Fault detection in telecom networks using bi-level federated graph neural networks," in *Proc. IEEE Int. Conf. Data Mining Workshops (ICDMW)*, Dec. 2023, pp. 1608–1617.

[39] J. You, J. Leskovec, K. He, and S. Xie, "Graph structure of neural networks," in *Proc. ICML*, 2020, pp. 1–11.

[40] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," in *Proc. NeurIPS*, 2020.

[41] Y. Ma, X. Liu, N. Shah, and J. Tang, "Is homophily a necessity for graph neural networks?," 2021, *arXiv:2106.06134*.

[42] J. Goh, S. Adepu, K. N. Junejo, and A. P. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. Int. Conf. Crit. Inf. Infrastructures Secur.*, 2017, pp. 88–99.

[43] A. Putina et al., "Telemetry-based stream-learning of BGP anomalies," in *Proc. Workshop Big Data Analytics Mach. Learn. Data Commun. Netw.*, Aug. 2018, pp. 15–20.

[44] C.-D. Phung, N.-E.-H. Yellas, S. B. Ruba, and S. Secci, "An open dataset for beyond-5G data-driven network automation experiments," in *Proc. 1st Int. Conf. 6G Netw. (6GNet)*, Jul. 2022, pp. 1–4.

[45] Prometheus. (Mar. 2024). *Prometheus Metric Website*. [Online]. Available: https://prometheus.io/docs/practices/naming/

[46] T. Feltin, P. Foroughi, W. Shao, F. Brockners, and T. H. Clausen, "Semantic feature selection for network telemetry event description," in *Proc. NOMS - IEEE/IFIP Netw. Operations Manage. Symp.*, Apr. 2020, pp. 1–6.

[47] V. Kalofolias, "How to learn a graph from smooth signals," in *Proc. PMLR*, 2016, pp. 920–929.

[48] G. Mateos, S. Segarra, A. G. Marques, and A. Ribeiro, "Connecting the dots: Identifying network structure via graph signal processing," *IEEE Signal Process. Mag.*, vol. 36, no. 3, pp. 16–43, May 2019.

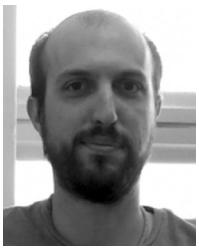
[49] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. NeurIPS*, vol. 26, 2013, pp. 3111–3119.

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.

[50] H. C. Wu, R. W. P. Luk, K. F. Wong, and K. L. Kwok, "Interpreting TF-IDF term weights as making relevance decisions," *ACM Trans. Inf. Syst.*, vol. 26, no. 3, pp. 1–37, Jun. 2008.
[51] J. Paparrizos, P. Boniol, T. Palpanas, R. S. Tsay, A. Elmore, and M. J. Franklin, "Volume under the surface: A new accuracy evaluation measure for time-series anomaly detection," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2774–2787, Jul. 2022.
[52] T. Lee, J. Gottschlich, N. Tatbul, E. Metcalf, and S. Zdonik, "Precision and recall for range-based anomaly detection," 2018, *arXiv:1801.03175*.
[53] N. Meinshausen and P. Bühlmann, "High-dimensional graphs and variable selection with the lasso," *Ann. Statist.*, vol. 34, no. 3, Jun. 2006.



Killian Cressant received the M.Sc. degree in computer science engineering from Telecom Nancy University, France, in 2023. He is currently pursuing the Ph.D. degree with the Division of Network and Connected Object, National Conservatory of Arts and Crafts (CNAM), Paris, France. His research interests include anomaly detection and graph neural networks and 5G networks.



Federico Larroca received the degree in telecommunication engineering from the Universidad de la República, Montevideo, Uruguay, in 2006, and the Ph.D. degree in computer science and networking from Telecom ParisTech, Paris, France, in December 2009, under the advisoring of Prof. Jean-Louis Rougier. He was a Research Engineer (Post-Doctoral) with Telecom ParisTech (formerly ENST) from January to March 2010. From 2004 to 2011, he held a Teaching Assistant position with the Universidad de la República. He is currently an Assistant Professor with the Engineering School, Universidad de la República. His research interests include the analysis and modeling of communication systems and development on software defined radio.



Stefano Secci received the M.Sc. degree in telecommunications engineering from the Politecnico di Milano, Italy, and the joint Ph.D. degree in networking from Telecom ParisTech, France, and the Politecnico di Milano. He was an Associate Professor with LIP6, Sorbonne University, from 2010 to 2018. He has been a Professor with CNAM, Cedric, Paris, France, since 2018. For more information, please visit: http:/cedric.cnam.fr/ seccis



Pedro Braconnot Velloso (Member, IEEE) received the B.Sc. and M.Sc. degrees in electrical engineering from the Universidade Federal do Rio de Janeiro, Rio de Janeiro, Brazil, in 2001 and 2003, respectively, and the Ph.D. degree from Université Pierre et Marie Curie (Paris 6), Paris, France, in 2008. He was a Research Engineer with Bell Labs, France. He was an Associate Professor with the Electronic Engineering Department, Universidade Federal do Rio de Janeiro. He is currently an Associate Professor with the Conservatoire National des Arts et Métiers (CNAM), Paris. His research interests include data science, distributed applications, wireless communications, and security.

Authorized licensed use limited to: UNIVERSITA TRENTO. Downloaded on November 04,2026 at 16:15:22 UTC from IEEE Xplore. Restrictions apply.